WARWICK
THE UNIVERSITY OF WARWICK

Lecture 2
Text Preprocessing

Gabriele Pergola

Natural Language Processing
CS918

January 2026

Why Text Preprocessing

- Inconsistent use of words:

  UK / U.K. / uk / u.k.

- Linguistic variations with similar meanings

  I am **happy** today.

  I am **happier** than yesterday.

For sentiment analysis, maybe all we need to know is that both have the word “happy”, irrespective of the variation.

Typical Text Preprocessing Steps

- Tokenisation
  - Segmenting sentences in running text
  - Segmenting/tokenising words in running text
- Stopword removal
- Normalising word formats – lemmatisation and stemming

Tokenisation

Tokenisation

- **Corpus:** collection or dataset of text or speech. One or more documents, e.g. corpus with all of Shakespeare's works.

- We can split each document/text into **sentences**, and these sentences into **words**.

- **Sentences:** shortest sequence of words that are grouped together to convey some grammatically correct self-contained meaning.
  - How do we split a text into sentences?
    - For practical purposes, sequence between full stops or "?|!|:|;"
    - We can do more advance segmentation, e.g. break long sentences with conjunctions like "and" or "or"

Tokenisation

Corpus

Document

Paragraph

Sentence

Token

Sentence Segmentation

- !, ? are relatively unambiguous

- But period "." is much more ambiguous
  - Sentence boundary
  - Abbreviations like Inc., etc. or PhD.
  - Numbers like .02% or 4.3

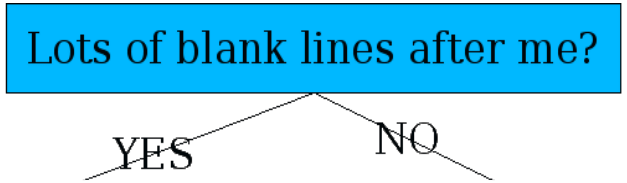
Sentence Segmentation…

- So how do we deal with this ambiguity?

- We can build a **binary classifier**:
  - Look at occurrences of '.'
  - Classifies ***EndOfSentence*** (EOS) vs ***NotEndOfSentence***. How?
    - Hand-written rules (if-then) → Example: **Decision Tree**
    - Regular expressions
    - Machine learning

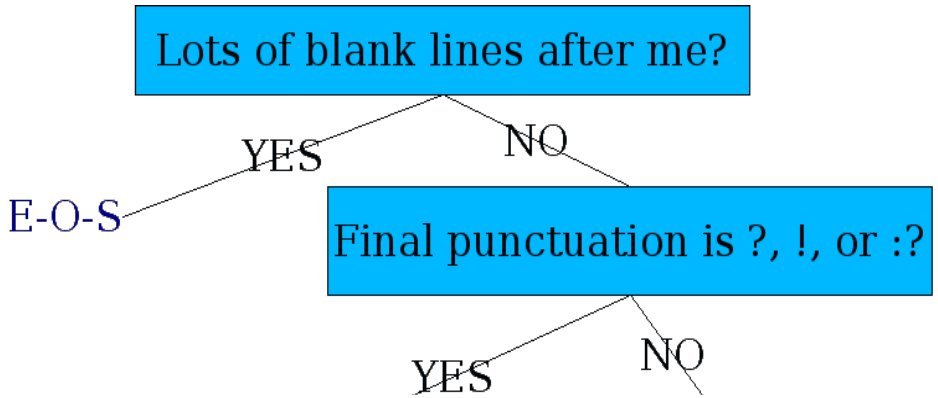
Sentence Segmentation: a Decision Tree

- Deciding if a **word** is at the end of a sentence



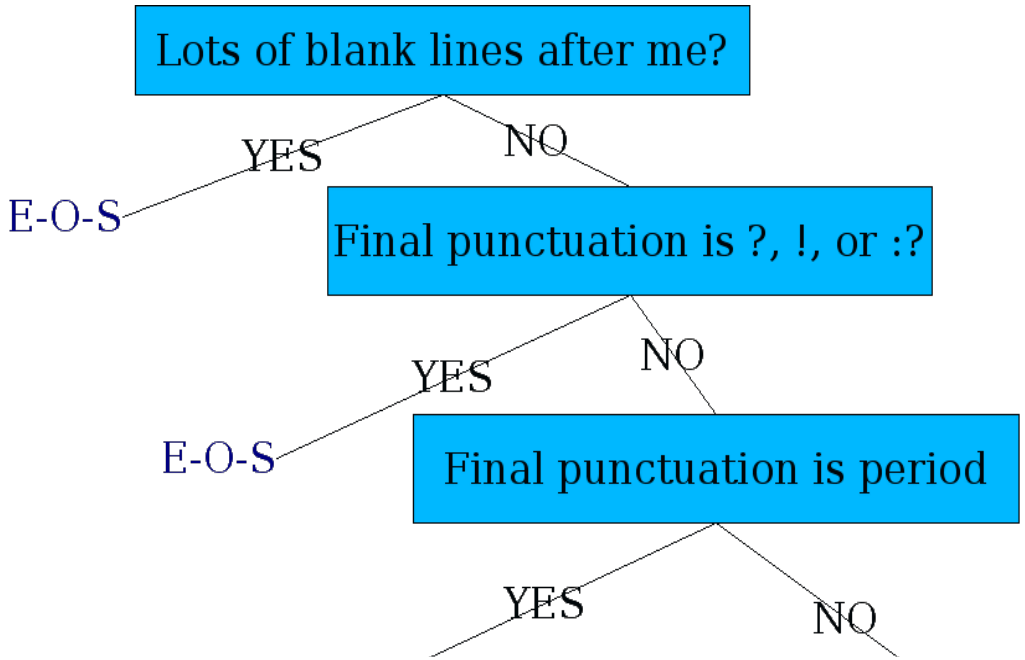
Sentence Segmentation: a Decision Tree

- Deciding if a **word** is at the end of a sentence



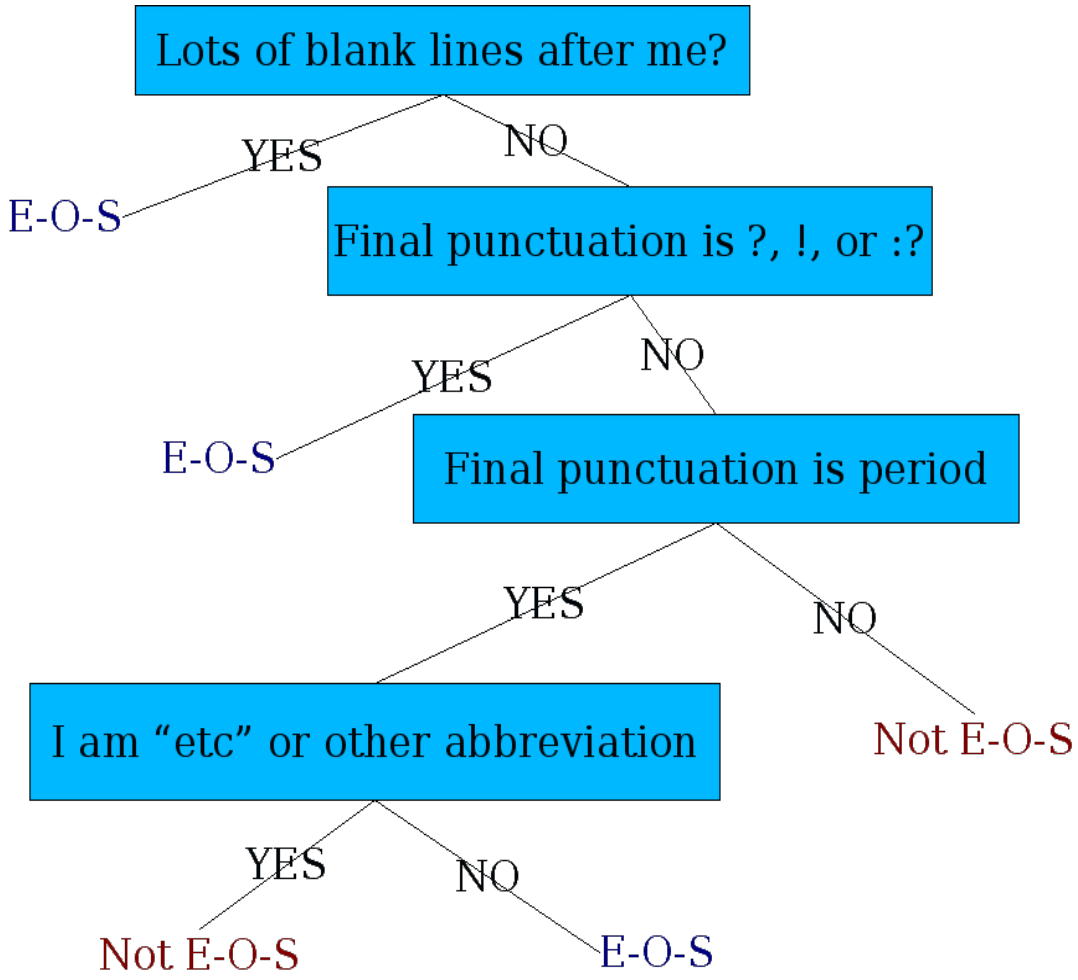
Sentence Segmentation: a Decision Tree

- Deciding if a **word** is at the end of a sentence



Sentence Segmentation: a Decision Tree

- Deciding if a **word** is at the end of a sentence



More features:

- Is the word following the period **uppercased**?
- What is the **length** of the word preceding the period?
- Are there **more periods** following? e.g. an ellipsis.
- Is there a **space** after the period?

Stopword Removal

Stopword Removal

- Some words are more **meaningful** than others

I am excited to be a member of Team GB!

Words like **"to", "be", "a", "of" are not very meaningful** for some analyses

- We call them "stop words",
  i.e. most commonly used words that are not informative for some tasks.
- If they're not useful for our task, we may remove them.
- Natural Language Toolkit (NLTK) provides a list of stop words.

How Many Words in a Sentence?

- It can be as simple as **counting** the elements we get after **splitting a text by spaces**, but it depends.

- How many words in the following?

My *cat is different from other cats.*

Words and Lemmas

- *My **cat** is different from other **cats**.*

- Cat and cats both have the same **lemma** (cat), but two different **wordforms**:
  - Cat: cat (lemma)
  - Cats: cat (lemma) + s (suffix)

How Many Words?

- **Type:** a unique element of the vocabulary.
- **Token:** an instance of that type in the running text.

The house on the hill is the best

8 tokens.
6 types: *the, house, on, hill, is, best*
(3 of the tokens belong the same type, '*the*')

If we remove stop words (*on*, *the*), these numbers will decrease.

Corpora

- **N** = number of tokens
- **V** = vocabulary = set of types
  - **|V|** is the size of the vocabulary

1. The "Standard" Era (Until 2022)

	Tokens = N	**Types = |V|**
Switchboard phone conversations	2.4 million	20 thousand
Shakespeare	884,000	31 thousand
Google N-grams	1 trillion	13 million

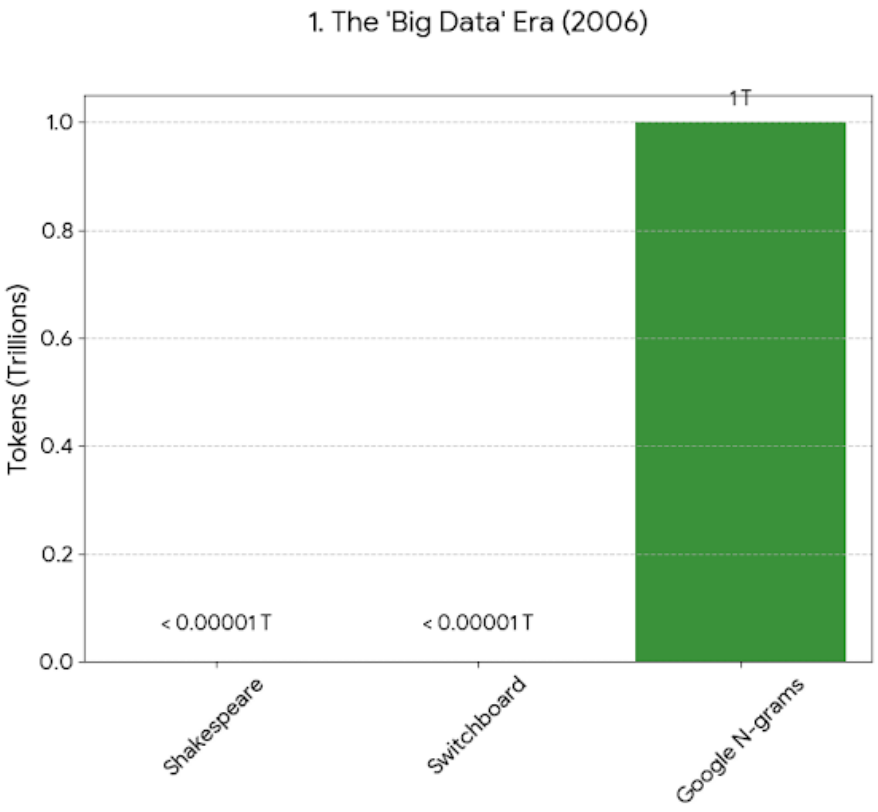
Corpora

- **N** = number of tokens
- **V** = vocabulary = set of types
  - **|V|** is the size of the vocabulary

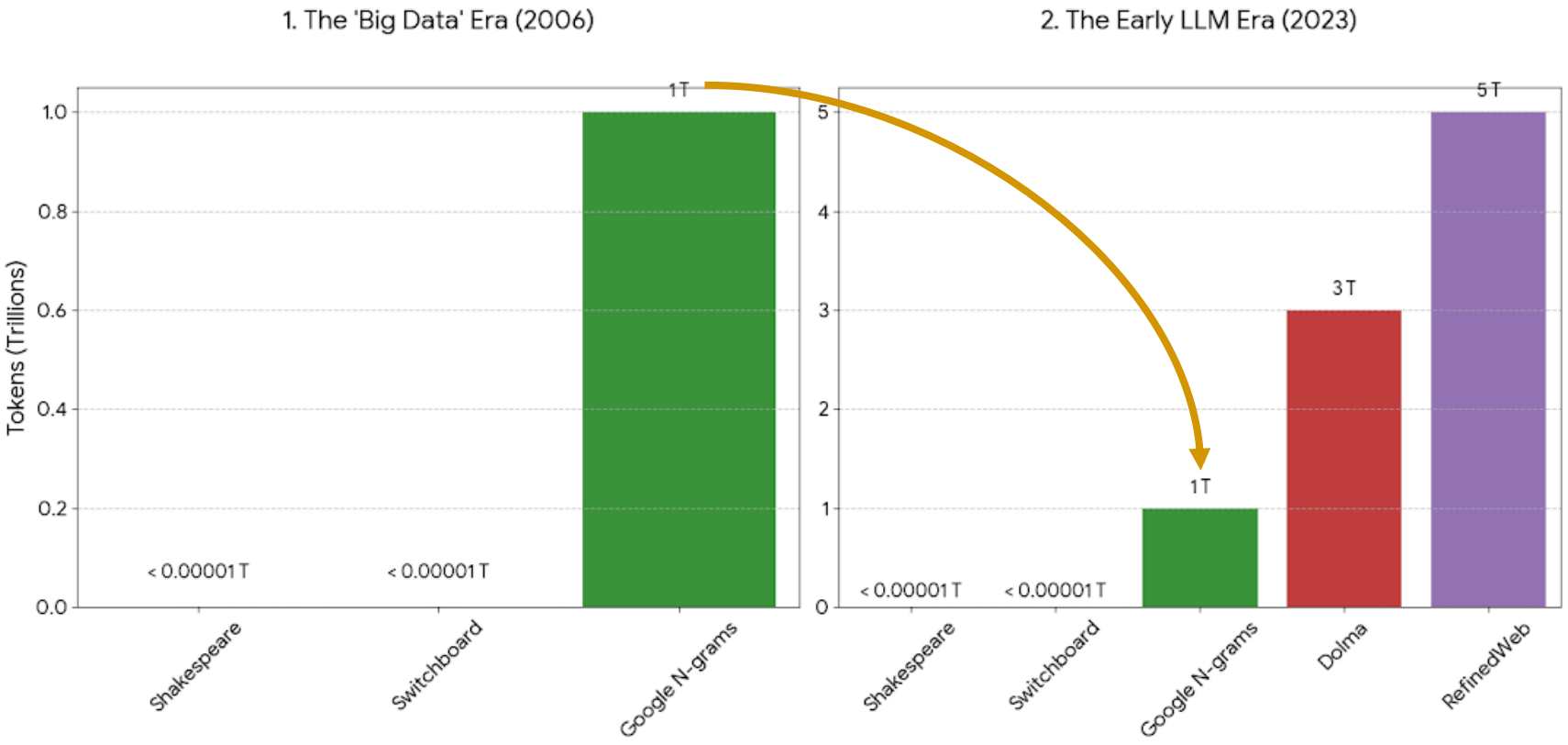
2. The "Trillion Token" Era (2023–Present)

	Tokens = N	Types = |V|
RefinedWeb (May23)	~5 T	-
Dolma (Aug23)	3 T	-
RedPajam v2 (Oct23)	30 T	-
FineWeb (May24)	15 T	-

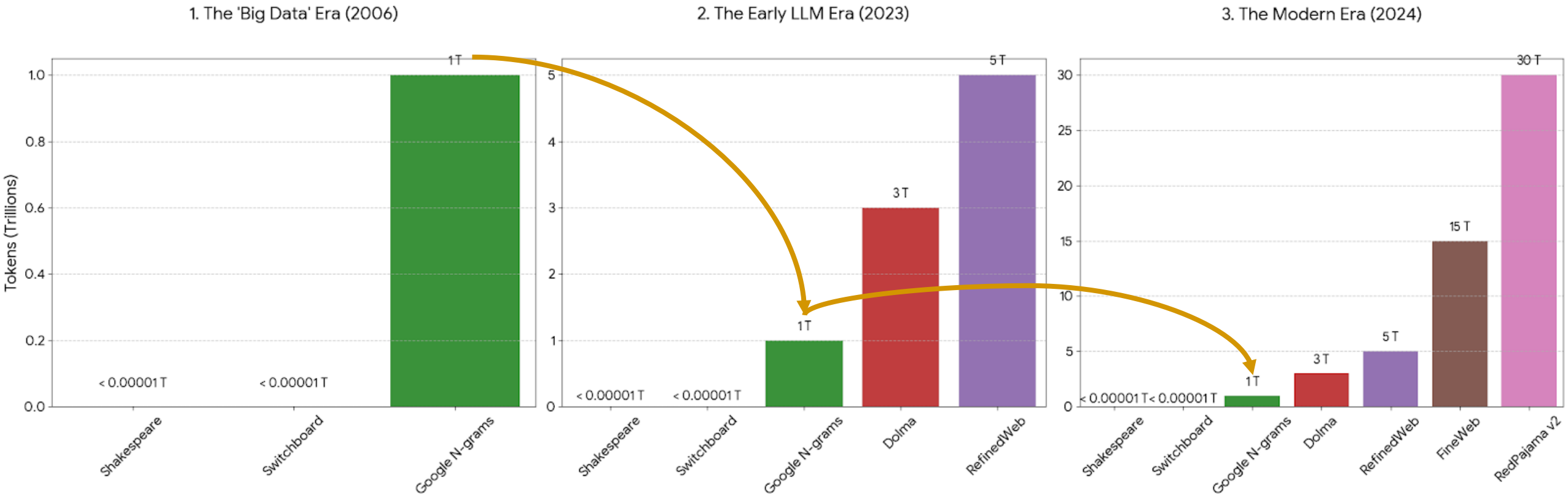
Corpora


1. The 'Big Data' Era (2006)
Tokens (Trillions)
1.0
0.8
0.6
0.4
0.2
0.0
1T
< 0.00001 T
< 0.00001 T
Shakespeare
Switchboard
Google N-grams

Corpora


1. The 'Big Data' Era (2006)
2. The Early LLM Era (2023)
Tokens (Trillions)
1.0
0.8
0.6
0.4
0.2
0.0
< 0.00001 T
< 0.00001 T
1 T
5
4
3
2
1
0
< 0.00001 T
< 0.00001 T
1 T
3 T
5 T
Shakespeare
Switchboard
Google N-grams
Shakespeare
Switchboard
Google N-grams
Dolma
RefinedWeb

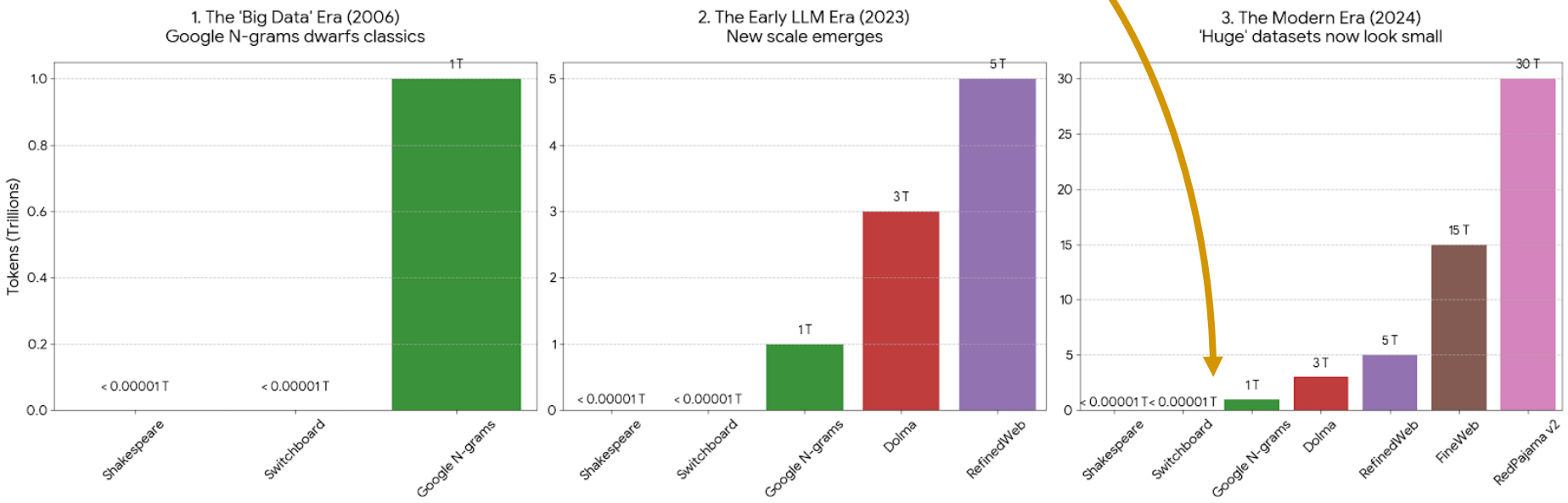
Corpora


1. The 'Big Data' Era (2006)
2. The Early LLM Era (2023)
3. The Modern Era (2024)
Tokens (Trillions)
1.0
0.8
0.6
0.4
0.2
0.0
< 0.00001 T
< 0.00001 T
1 T
Shakespeare
Switchboard
Google N-grams
5
4
3
2
1
0
< 0.00001 T
< 0.00001 T
1 T
3 T
5 T
Shakespeare
Switchboard
Google N-grams
Dolma
RefinedWeb
30
25
20
15
10
5
0
< 0.00001 T
< 0.00001 T
1 T
3 T
5 T
15 T
30 T
Shakespeare
Switchboard
Google N-grams
Dolma
RefinedWeb
FineWeb
RedPajama v2

How Many Words?

- How many tokens to train GPT-3?

300B Tokens



Language Models are Few-Shot Learners, NeurIPS 2020

Word Tokenisation

- The Complete Works of William Shakespeare (`shakes.txt`, available on module website):

Open ▾ shakes.txt ~/Desktop

```
The Project Gutenberg EBook of The Complete Works of William Shakespeare, by
William Shakespeare

This eBook is for the use of anyone anywhere at no cost and with
almost no restrictions whatsoever.  You may copy it, give it away or
re-use it under the terms of the Project Gutenberg License included
with this eBook or online at www.gutenberg.org

** This is a COPYRIGHTED Project Gutenberg eBook, Details Below **
**     Please follow the copyright guidelines in this file.     **

Title: The Complete Works of William Shakespeare

Author: William Shakespeare

Posting Date: September 1, 2011 [EBook #100]
Release Date: January, 1994

Language: English


*** START OF THIS PROJECT GUTENBERG EBOOK COMPLETE WORKS--WILLIAM SHAKESPEARE ***



Produced by World Library, Inc., from their Library of the Future
```

Simple Tokenisation in Unix

```
> tr -sc 'A-Za-z' '\n' < shakes.txt
```

- Having shakes.txt as input (< shakes.txt)
- Convert all non-alphabetic characters (-sc ‘A-Za-z’)
  - ’A-Za-z’ means alphabetic
  - -c option complements to non-alphabet
  - -s option squeezes all sequences into a single character
- Into new lines (‘\n’)

Step 1: Tokenising

```
> tr -sc 'A-Za-z' '\n' < shakes.txt | head
The
Project
Gutenberg
EBook
of
The
Complete
Works
of
...
```

Sorting

```
> tr -sc 'A-Za-z' '\n' < shakes.txt | sort | head
a
a
a
a
a
a
a
a
a
...
```

Step 3: Counting

```
> tr -sc 'A-Za-z' '\n' < shakes.txt | sort | uniq -c | head
  12851 a
   1949 A
     25 Aaron
     72 AARON
      1 abaissiez
     10 abandon
      2 abandoned
      2 abase
      1 abash
...
```

Step 3: Sort by count

```
> tr -sc 'A-Za-z' '\n' < shakes.txt | sort | uniq -c | sort -r -n | head
  23455 the
  22225 I
  18715 and
  16433 to
  15830 of
  12851 a
  12236 you
  10840 my
  10074 in
   8954 d→ what happened here?
...
```

Step 4: Lowercasing text

```
> tr -sc 'A-Za-z' '\n' < shakes.txt | tr 'A-Z' 'a-z' | sort | uniq -c
| sort -r -n | head
  27843 the
  26847 and
  22538 i
  19882 to
  18307 of
  14800 a
  13928 you
  12490 my
  11563 that
  11183 in
...
```

Issues in Tokenisation

- England’s capital → England, Englands, England’s
- what’re, I’m, isn’t → What are, I am, is not
- Hewlett-Packard → Hewlett Packard
- state-of-the-art → state of the art ?
- lower-case, lowercase, lower case
- Leamington Spa → one token or two?
- U.K./UK, U.S.A./USA

Penn Treebank Tokenisation

- “The Leamington Spa-based restaurant,” they said, “doesn’t charge £10”.

“ The Leamington **Spa-based** restaurant **,** “ they said **,** “ does **n’t** charge £ 10 “ .

Uses regular expressions:

- ftp://ftp.cis.upenn.edu/pub/treebank/public_html/tokenization.html
- https://www.nltk.org/_modules/nltk/tokenize/treebank.html

Tokenisation: Language Issues

- French:
  - ***L'ensemble*** → one token or two?
  - L ? L' ? Le ?
  - We want *l'ensemble* to match other instances of ensemble

- German noun compounds are not segmented:
  - ***Lebensversicherungsgesellschaftsangestellter***
  - 'life insurance company employee'
  - German information retrieval needs *compound splitter*

Tokenisation in Chinese

- Also called **Word Segmentation**

- Chinese words are composed of characters:
  - Characters are generally 1 syllable and 1 morpheme (single unit of meaning)
  - Average word is 2.4 characters long

- Standard baseline segmentation algorithm:
  - **Maximum Matching** (also called Greedy)

Maximum Matching Algorithm (also called Greedy)

- Given a wordlist (dictionary) of Chinese, and a string as input:

  1) Start a pointer at the beginning of the string
  2) Find the longest word in dictionary that matches the string starting at pointer
  3) Move the pointer over the word in string
  4) Go to 2

Maximum Matching Algorithm: **Example in English**

- Thetabledownthere

Longest dictionary word from the beginning is ‘theta’, but we wanted ‘the’.

We get ‘Theta bled own there’
We probably wanted ‘The table down there’ though!

It’s quite a **bad algorithm for English**!

Maximum Matching Algorithm: **Example in English**

- But it’s actually very good for Chinese:
  - 莎拉波娃现在居住在美国东南部的佛罗里达。
  - 莎拉波娃 现在 居住 在 美国 东南部 的 佛罗里达

Normalisation

Normalisation

- 2 types of normalisation. Let's think of a Google search query.
  - Symmetric normalisation:
    - User searching for 'U.S.A.' or 'USA' or 'US' most likely looking for the same
  - Asymmetric normalisation:
    - User enters 'Windows', we give results for 'Windows' (operating system)
    - User enters 'windows', do we give results for both 'Windows' and 'windows'?

Case Folding

- Often the case is not meaningful, e.g. ‘the’ vs ‘The’.
  - We may reduce all to **lowercase**.
  - Possible exception: upper case in mid-sentence?
    - e.g., **General Motors**

- But the case is sometimes important!
  - e.g. **US** vs. **us**

Lemmatisation and Stemming

- In both cases, we aim to **reduce vocabulary size**.
  - e.g. ‘cars’ and ‘car’ will both become ‘car’

- **Lemmatisation:** finding dictionary headword form

- **Stemming:** finding the stem by stripping off suffixes, usually using regular expressions

Lemmatisation

- Reduce inflections or variant forms to headword form:
  - am, are, is → be
  - car, cars, car's, cars' → car
  - those cars are really beautiful → those car be really beautiful

- PRO: we end up getting dictionary words
- CON: costly, need to infer the meaning of each word
  - Reading (verb) → read
  - Reading (city) → Reading

Stemming

- Reduce by **following certain rules** (e.g. regular expressions):
  - am, are, is → am, ar, is
  - car, cars, car's, cars' → car, car, car ' s, car '
  - those cars are really beautiful → those car ar realli beauti

- CON: It does shorten words and reduce vocabulary, however **not always leading to dictionary words**!

- PRO: It's **faster than a lemmatiser**.

Porter's Stemmer

Step 1a

sses	→ ss	posesses	→ posess
ies	→ i	ponies	→ poni
ss	→ ss	posess	→ posess
s	→ ø	cats	→ cat

Step 1b

(*v*)ing	→ ø	walking	→ walk
		sing	→ sing
(*v*)ed	→ ø	plastered	→ plaster

…

https://tartarus.org/martin/PorterStemmer/

Porter's Stemmer

- (*v*)ing → ø
- having → hav, living → liv, studying → study ✓
- king → ø, sing → ø, thing → ø ✓
- something → someth, morning → morn ✗

Lemmariser vs. Stemmer

for example compressed and compression are both accepted as equivalent to compress.

STEMMER:
for exampl compress and compress ar both accept as equival to compress

LEMMATISER:
for example compress and compress be both accept as equivalent to compress

Reading

- Jurafsky, Daniel, and James H. Martin. 2026. Speech and Language Processing. 3rd edition. **Chapter 2.2-2.5**.

- Bird Steven, Ewan Klein, and Edward Loper. Natural Language Processing with Python. O'Reilly Media, Inc., 2009. **Chapters 1-3**.

Live Quiz

SCAN ME

https://pollev.com/gabrieleperg177

Which of the following is an NLP application?

Predicting the genre of books

0%

Translate text from a language to another

0%

Summarising a text or article

0%

All of them

0%

Start the presentation to see live content. For screen share software, share the entire screen. Get help at **pollev.com/app**

What is the main aim in information retrieval?

Generating structured information from unstructured text

0%

Find relevant materials (e.g., documents) for a given query

0%

Summarised the information in a document

0%

Detect the syntactical information in a document

0%

Start the presentation to see live content. For screen share software, share the entire screen. Get help at **pollev.com/app**

What is information extraction?

Find relevant materials (e.g., documents) for a given query

0%

Generating structured information from unstructured text

0%

Detect the syntactical information in a document

0%

Summarise the information in a document

0%

Start the presentation to see live content. For screen share software, share the entire screen. Get help at **pollev.com/app**

Regular expressions are useful for:

Translating from one language to another

0%

Summarising text

0%

Specifying text patterns

0%

Performing stemming

0%

Start the presentation to see live content. For screen share software, share the entire screen. Get help at **pollev.com/app**

What is tokenisation?

Spliting the running text into text units

0%

Specifing patterns occurring in text

0%

Removing uninformative words

0%

Finding dictionary headword form

0%

Start the presentation to see live content. For screen share software, share the entire screen. Get help at **pollev.com/app**

Which of these techniques is used for the normalization of text?

Lemmatization

0%

Stemming

0%

Stop words removal

0%

All of them

0%

Start the presentation to see live content. For screen share software, share the entire screen. Get help at **pollev.com/app**

What stemming refers to in NLP?

Detecting the syntactical role of a word

0%

Splitting sentences into words

0%

Removing suffixes from the end of a word

0%

Mapping a word to its original root

0%

Start the presentation to see live content. For screen share software, share the entire screen. Get help at **pollev.com/app**

What is lemmatization?

Removing suffixes from the end of a word

0%

Splitting sentences into words

0%

Mapping a word to its original root

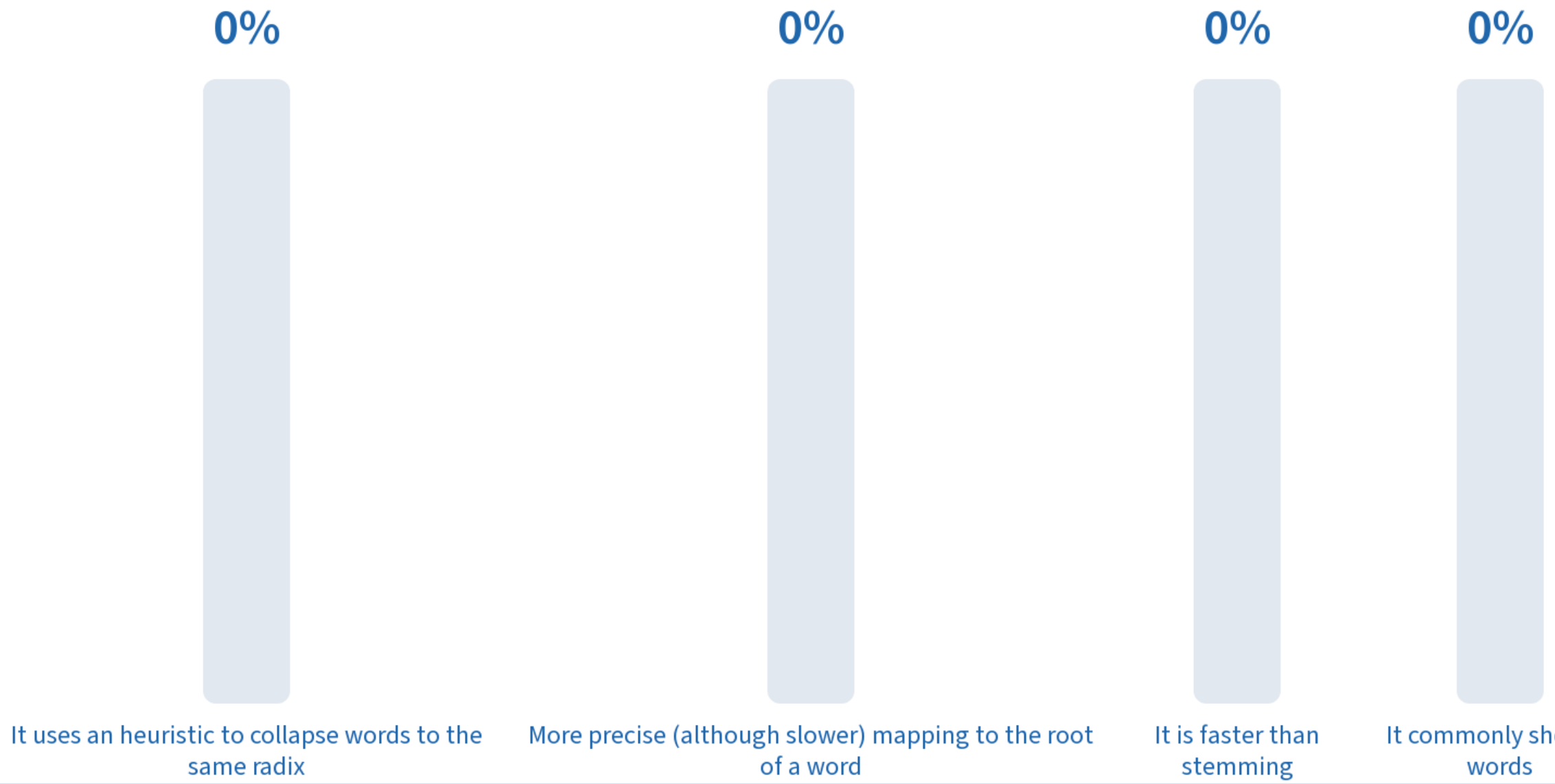
0%

Detecting the syntactical role of a word

0%

Start the presentation to see live content. For screen share software, share the entire screen. Get help at **pollev.com/app**

What's the advantage of lemmatizing over stemming?



Start the presentation to see live content. For screen share software, share the entire screen. Get help at **pollev.com/app**

How the words 'lovely' is converted to 'love'?

By lemmatization

0%

By tokenization

0%

By stemming

0%

None of the previous

0%

Start the presentation to see live content. For screen share software, share the entire screen. Get help at **pollev.com/app**

Stop words are:

Words that frequently found in a document

0%

Words that have no use in prediction

0%

Word that are not very informative

0%

All of them

0%

Start the presentation to see live content. For screen share software, share the entire screen. Get help at **pollev.com/app**